



CS 540 Introduction to Artificial Intelligence

Reinforcement Learning I

University of Wisconsin-Madison
Spring 2025

Announcements

- **Homework:**

- HW9 due on **Wednesday April 23rd at 11:59 PM**

- **Class roadmap:**

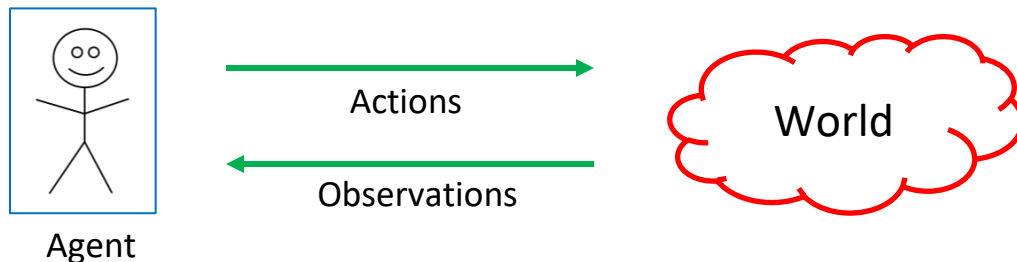
Introduction to Reinforcement Learning
Reinforcement Learning II
Advanced Search

Outline

- Introduction to reinforcement learning
 - Basic concepts, mathematical formulation, MDPs, policies.
- Learning policies
 - Q-learning, action-values, exploration vs exploitation.

Back to Our General Model

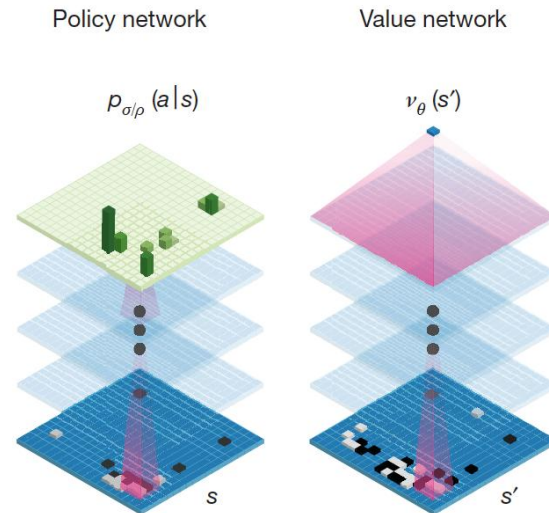
We have an **agent** **interacting** with the **world**



- Agent receives a reward based on state of the world
 - ★ – Goal: maximize reward / utility (\$\$\$)
 - Note: **data** consists of actions & observations
 - Compare to unsupervised learning and supervised learning

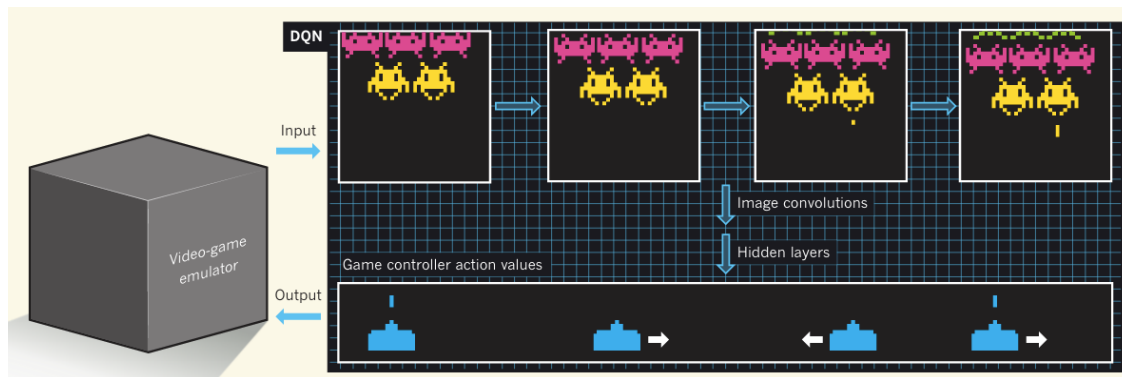
Examples: Gameplay Agents

AlphaZero:

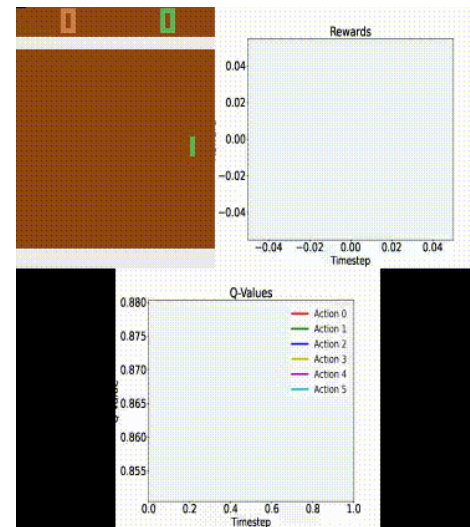


Examples: Video Game Agents

Pong, Atari



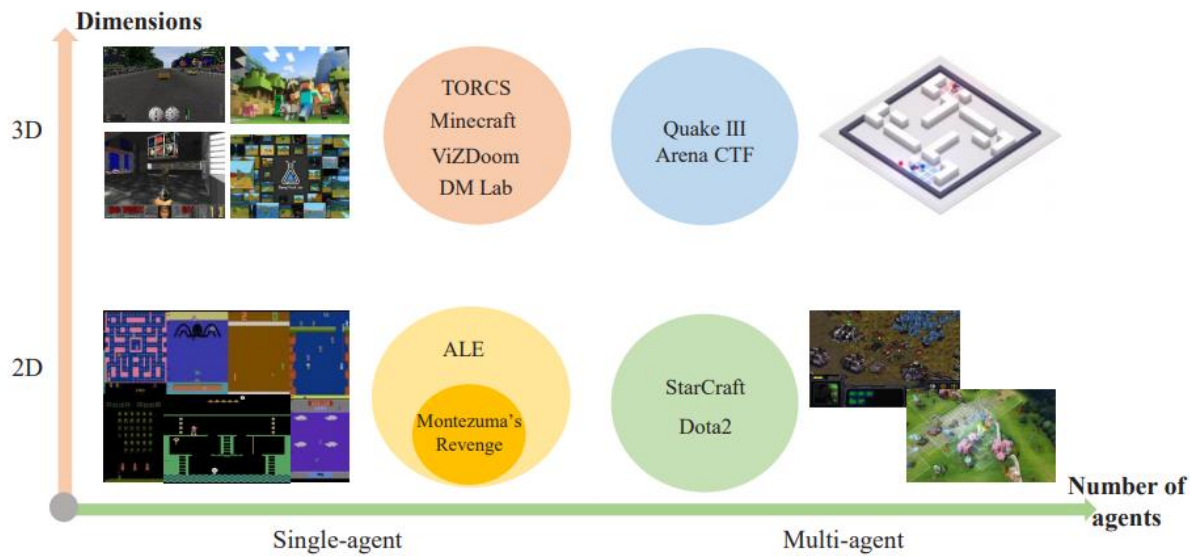
Mnih et al, "Human-level control through deep reinforcement learning"



[A. Nielsen](#)

Examples: Video Game Agents

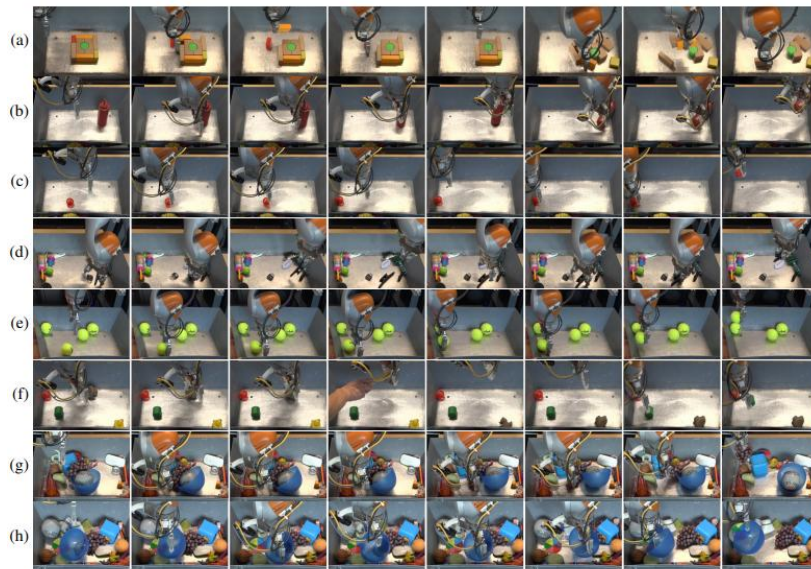
Minecraft, Quake, StarCraft, and more!



Shao et al, "A Survey of Deep Reinforcement Learning in Video Games"

Examples: Robotics

Training robots to perform tasks (e.g., grasp objects!)



Examples: Large Language Models

RL used to “align” model outputs to human preferences

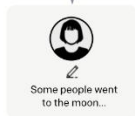
Step 1

Collect demonstration data, and train a supervised policy.

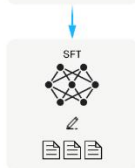
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



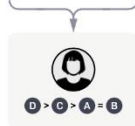
Step 2

Collect comparison data, and train a reward model.

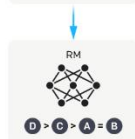
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



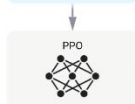
Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



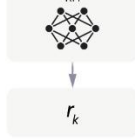
The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Unaligned Model

There are many ways to cheat in poker, including using marked cards, collusion, and using electronic devices to track other players' cards.

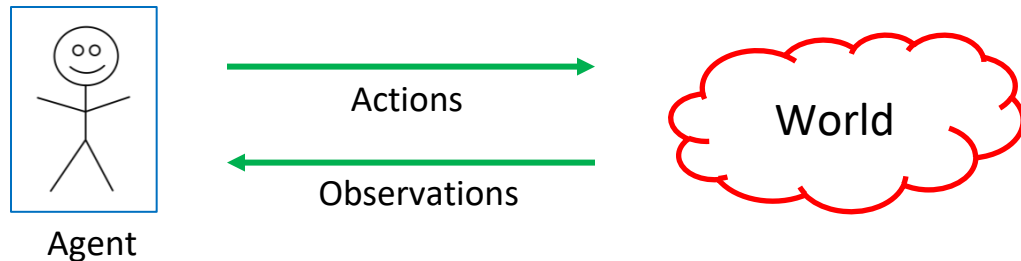
Aligned Model Post-RLHF

Cheating in poker is illegal and unethical, and can result in penalties such as fines or expulsion from the game. There are many ways to win at poker legitimately, such as developing a strong strategy, practicing, and analyzing your opponents' behavior.

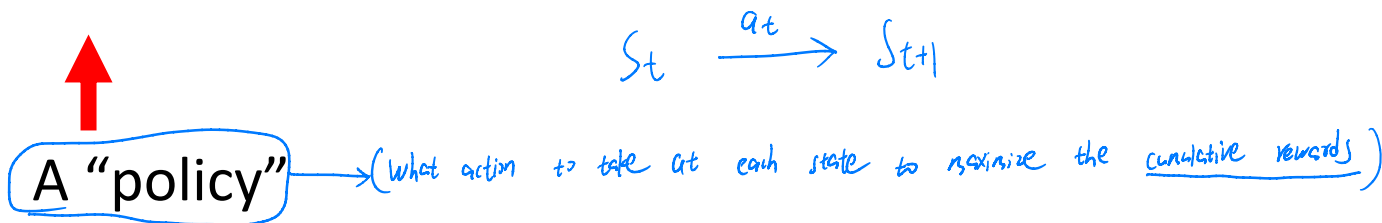
Building The Theoretical Model

Basic setup:

- Set of states, S
- Set of actions, A
- Information: at time t , observe state $s_t \in S$. Get reward r_t
- Agent makes choice $a_t \in A$. State changes to s_{t+1} , continue



★ Goal: find a map from **states to actions** that maximize rewards.



Markov Decision Process (MDP)

The formal mathematical model:

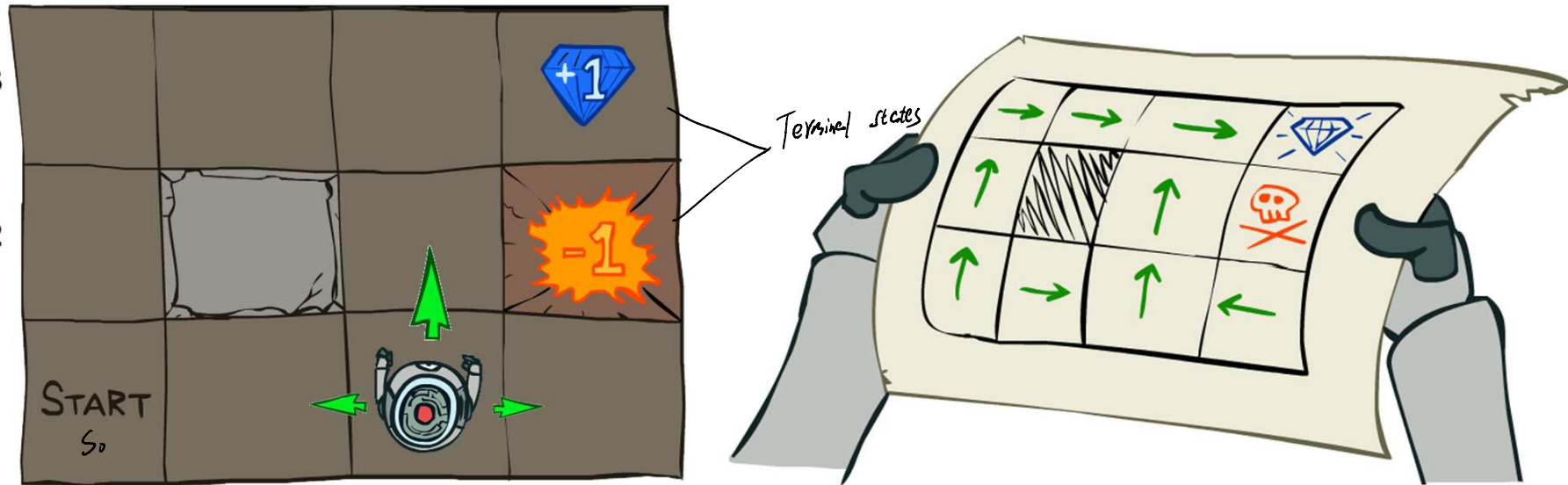
- **State set** S . Initial state s_0 . **Action set** A
- **Reward function:** $r(s_t)$
- ★ • **State transition model:** $P(s_{t+1} | s_t, a_t)$
 - Markov assumption: transition probability only depends on s_t and a_t , and not earlier history (previous actions or states)
- More generally: $r(s_t, a_t)$, potentially random
- ★ • **Policy:** $\pi(s) : S \rightarrow A$ action to take at a particular state

Our goal is to find a good π

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots$$

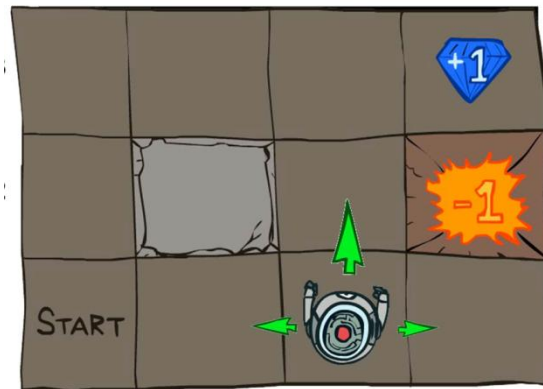
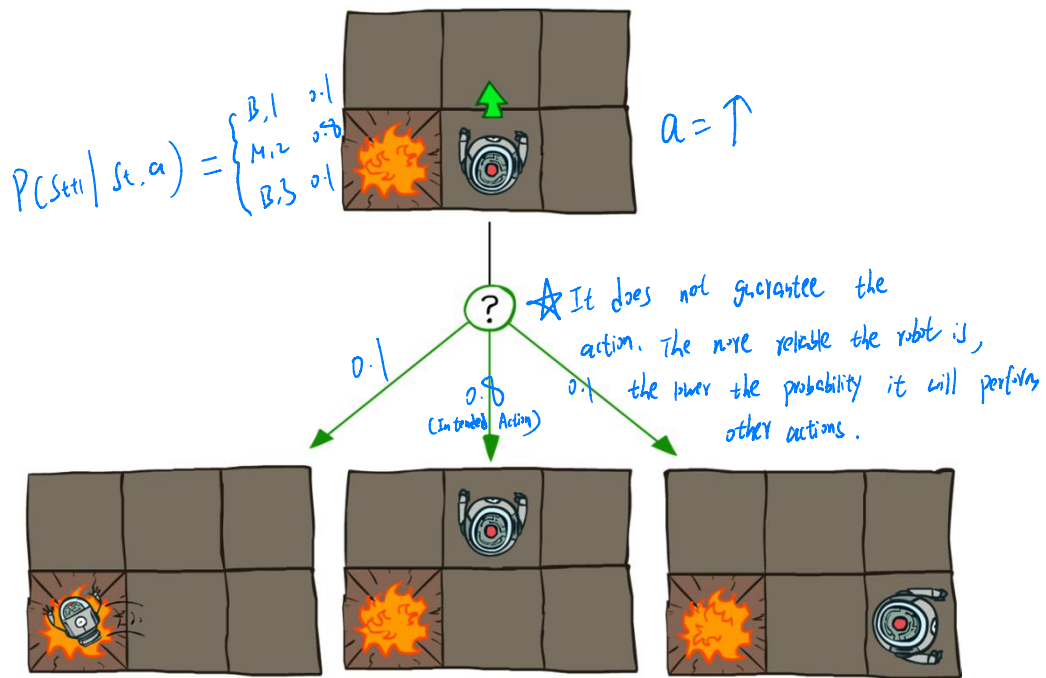
Example of MDP: Grid World

Robot on a grid; goal: find the best policy



Example of MDP: Grid World

Note: (i) Robot is unreliable (ii) Reach target fast

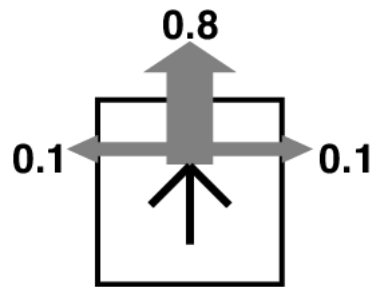
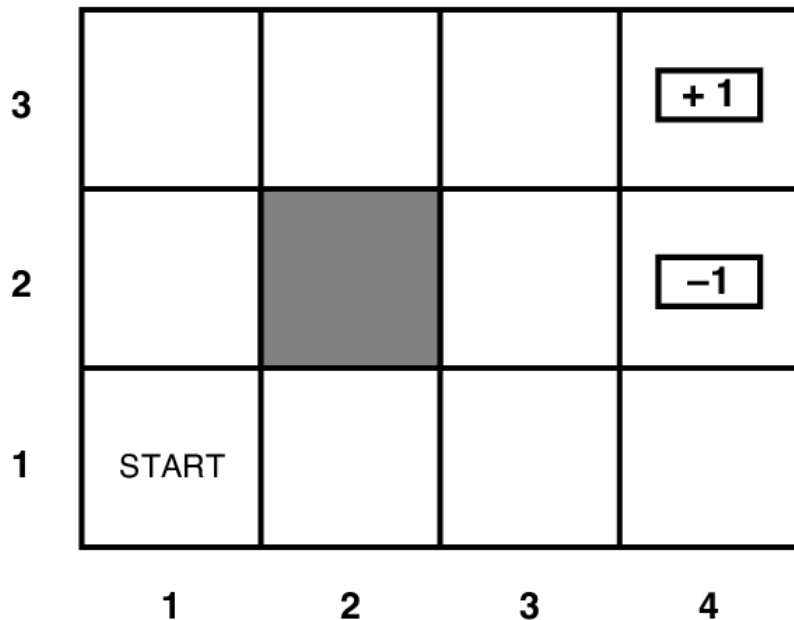


$r(s) = -0.04$ for every non-terminal state

We want a fast path to any terminal state

Grid World Abstraction

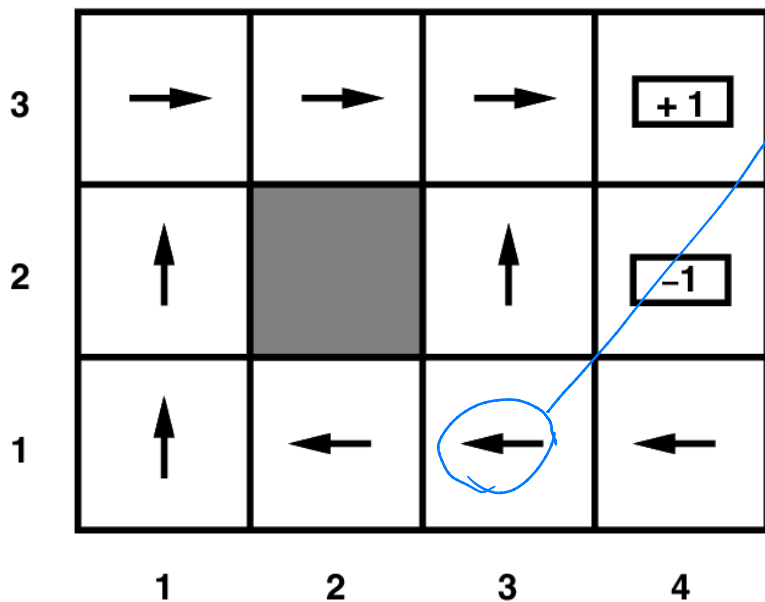
Note: (i) Robot is unreliable (ii) Reach target fast



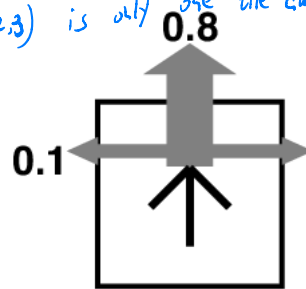
$r(s) = -0.04$ for every non-terminal state

Grid World Optimal Policy

Note: (i) Robot is unreliable (ii) Reach target fast



why? we could have gone upwards (better). This is because the field (2,3) is only one tile away from the -1. Although it is only 10% for entering that, we still'd like to avoid that and rather be safe to go around the grid.



$r(s) = -0.04$ for every non-terminal state

Back to MDP Setup

The formal mathematical model:

- **State set** S . Initial state s_0 . **Action set** A
 - **State transition model:** $P(s_{t+1} | s_t, a_t)$
 - Markov assumption: transition probability only depends on s_t and a_t , and not previous actions or states.
 - **Reward function:** $r(s_t)$
 - **Policy:** $\pi(s) : S \rightarrow A$ action to take at a particular state.
- How do we find the best policy?**
- 

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots$$

Reinforcement Learning Challenges

Credit-assignment: (Delayed reward)

- May take many actions before reward is received. Which ones were most important?
- Example: You study 15 minutes a day all semester. The morning of the final exam, you eat a bowl of yogurt. You receive an A on the final. Was it the studying or the yogurt that led to the A?

Exploration vs. Exploitation:

- Transition probabilities and reward may be unknown to the learner.
- Should you keep trying actions that led to reward in the past or try new actions that might lead to even more reward?

(Short-term reward and Long-term reward)

Break & Quiz

Q 1.1 Which of the following statement about MDP is **not** true?

- A. The reward function must output a scalar value
- B. The policy maps states to actions
- C. The probability of next state can depend on current and previous states 
- D. The solution of MDP is to find a policy that maximizes the cumulative rewards

Defining the Optimal Policy

For policy π , expected utility over all possible state sequences from s_0 produced by following that policy:

$$V^\pi(s_0) = \sum_{\text{sequences starting from } s_0} P(\text{sequence}) U(\text{sequence})$$

Utility of sequence

Value

Probability of sequence when following π

Called the value function (for π , s_0)



Discounting Rewards

① Avoid divergence ② Incentivize the short-term progress

Utility can add up the rewards over a sequence of states, but how should we treat the future?

- Solution: **discount** future rewards.

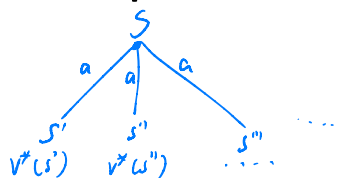
$$U(\underset{\substack{\parallel \\ U(\text{sequence})}}{s_0}, \underset{\substack{\parallel \\ \text{gamma} \cdot}}{s_1} \dots) = \underset{\substack{\parallel \\ \text{gamma} \cdot}}{r(s_0)} + \gamma \underset{\substack{\parallel \\ \text{gamma} \cdot}}{r(s_1)} + \gamma^2 \underset{\substack{\parallel \\ \text{gamma} \cdot}}{r(s_2)} + \dots = \sum_{t \geq 0} \gamma^t \underset{\substack{\parallel \\ \text{gamma} \cdot}}{r(s_t)}$$

- Discount factor γ between 0 and 1
 - Set according to how important **present** is versus **future**
 - Note: has to be less than 1 for convergence

From Value to Policy

Now that $V^\pi(s_0)$ is defined, what a should we take?

- First, let π^* be the **optimal** policy for $V^\pi(s_0)$, and $V^*(s_0)$ its expected utility.
Optimal scenario
- What's the expected utility following an action?
 - Specifically, action a in state s ?



$$\sum_{s'} P(s'|s, a) V^*(s')$$

(Average value I get from taking this a . And this is the best average cumulative reward by following the π^* . Any other policy will be worse than this)

All the states we could go to

Transition probability

Expected rewards

Slight Problem...

Now we can get the optimal policy by doing

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s, a) V^*(s')$$

*The action to give us the best value $\rightarrow$ best policy π^**

- So we need to know $V^*(s)$ (and P).
 - But it was defined in terms of the optimal policy!
 - So we need some other approach to get $V^*(s)$.
 - Instead, learn about the utility of actions directly.

Bellman Equation

Let's walk over one step for the value function:

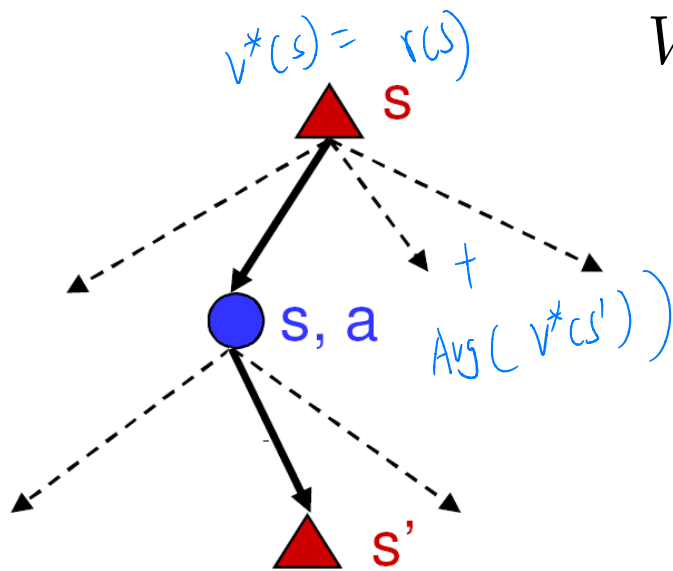
$$V^*(s) = \underset{\substack{\uparrow \\ \text{Current state} \\ \text{reward}}}{r(s)} + \gamma \overset{\text{(Recursion)}}{\max_a \underbrace{\sum_{s'} P(s'|s, a) V^*(s')}_{\substack{\text{Discounted expected} \\ \text{future } \textcolor{green}{\text{rewards}}}}}$$

- Richard Bellman: inventor of dynamic programming



Bellman Equation

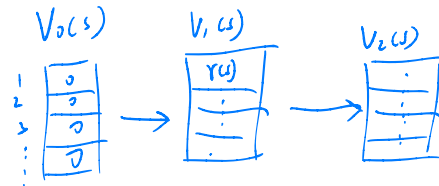
Let's walk over one step for the value function:



Credit L. Lazbenik

$$V^*(s) = \underset{\text{Current state reward}}{\underbrace{r(s)}} + \gamma \underbrace{\max_a \sum_{s'} P(s'|s, a) V^*(s'))}_{\text{Discounted expected future rewards}}$$

Value Iteration



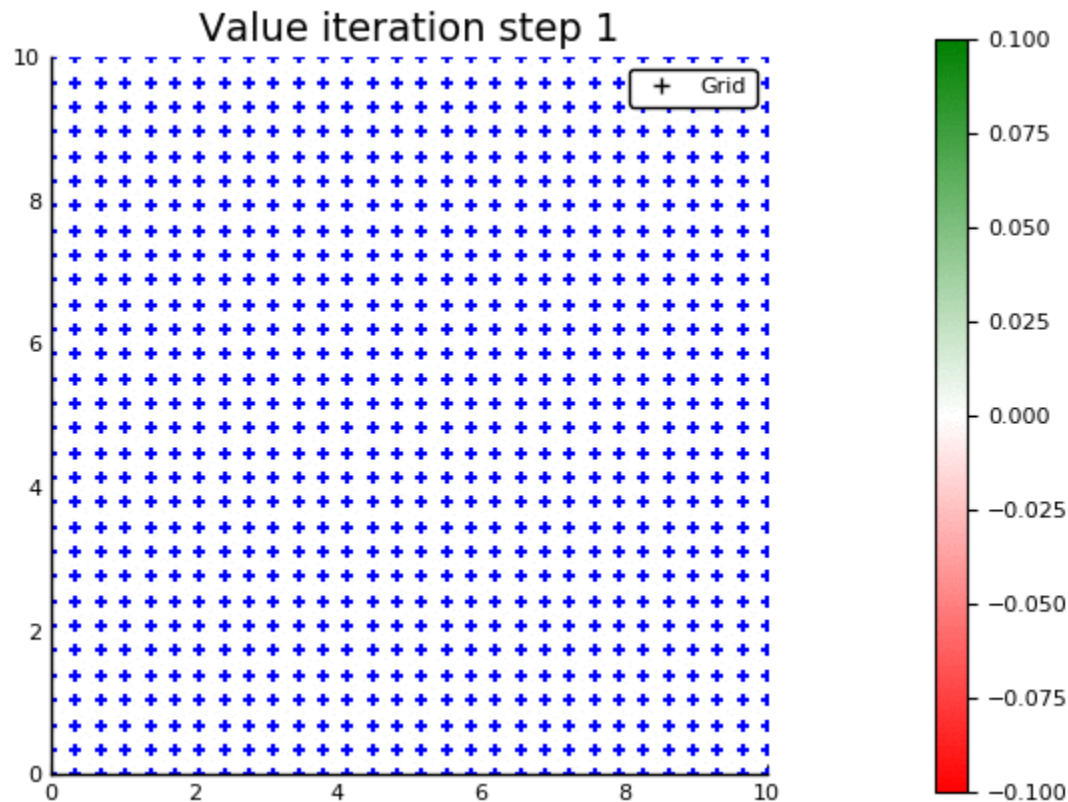
Q: how do we find $V^*(s)$?

- Why do we want it? Can use it to get the best policy
- Know: reward $r(s)$, transition probability $P(s' | s, a)$
 - Knowing r and P is the “planning” problem. In reality r and P must be estimated from interactions : “reinforcement learning”
- Also know $V^*(s)$ satisfies Bellman equation (recursion above)

A: Use the property. Start with $V_0(s) = 0$. Then, update

Value Iteration $\leftarrow V_{i+1}(s) = r(s) + \gamma \max_a \sum_{s'} P(s' | s, a) V_i(s')$

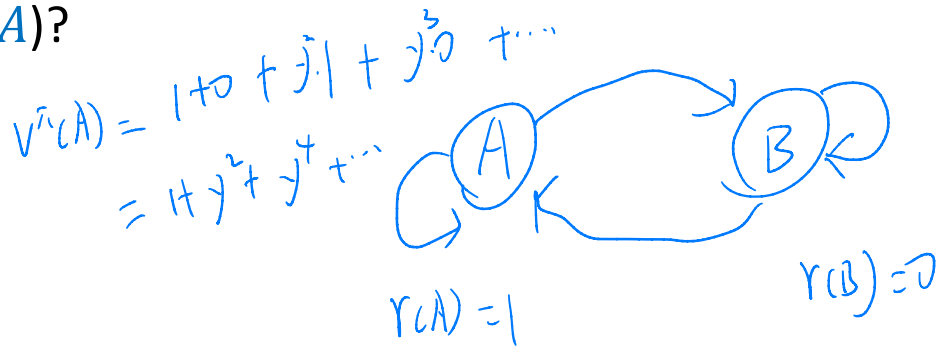
Value Iteration: Demo



Break & Quiz

Q 2.1 Consider an MDP with 2 states $\{A, B\}$ and 2 actions: “**stay**” at current state and “**move**” to other state. Here, transitions are deterministic. Let r be the reward function such that $r(A) = 1$, $r(B) = 0$. Let γ be the discounting factor. Let π : $\pi(A) = \pi(B) = \text{move}$ (i.e., an “always move” policy). What is the value function $V^\pi(A)$?

- A. 0
- B. $1 / (1 - \gamma)$
- C. $1 / (1 - \gamma^2)$
- D. 1



Handwritten optimal policy:

$$\pi^*(A) = \text{STAY}$$
$$\pi^*(B) = \text{MOVE}$$

Q-Learning

- Our **next** reinforcement learning algorithm.
- Does not require knowing r or P . Learn from data of the form: $\{(s_t, a_t, r_t, s_{t+1})\}$.
- Learns an action-value function $Q^*(s, a)$ that tells us the expected value of taking a in state s .
 - Note: $V^*(s) = \max_a Q^*(s, a)$.
- Optimal policy is formed as $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$

The $Q^*(s,a)$ function

- Starting from state s , perform (perhaps suboptimal) action a . THEN follow the optimal policy

$$Q^*(s, a) = r(s) + \gamma \sum_{s'} P(s'|s, a) V^*(s')$$

- Equivalent to

$$Q^*(s, a) = r(s) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

Q-Learning Iteration

How do we get $Q(s, a)$?

- Iterative procedure

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r(s_t) + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

Learning rate



Idea: combine old value and new estimate of future value.

Note: We are using a policy to take actions; based on the estimated Q!

Q-Learning

Estimate $Q^*(s, a)$ from data $\{(s_t, a_t, r_t, s_{t+1})\}$:

1. Initialize $Q(.,.)$ arbitrarily (eg all zeros)
 1. Except terminal states $Q(s_{\text{terminal}},.)=0$
2. Iterate over data until $Q(.,.)$ converges:

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha(r_t + \gamma \max_b Q(s_{t+1}, b))$$



Learning rate

Q-Learning

Estimate $Q^*(s, a)$ from data $\{(s_t, a_t, r_t, s_{t+1})\}$:

1. Initialize $Q(.,.)$ arbitrarily (eg all zeros)
 1. Except terminal states $Q(s_{\text{terminal}},.)=0$
2. Iterate over data until $Q(.,.)$ converges:

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha(r_t + \gamma \max_b Q(s_{t+1}, b))$$

Idea: update is an empirical version of our Q table
recursion:

$$Q^*(s, a) = r(s) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

Exploration Vs. Exploitation

General question!

- **Exploration:** take an action with unknown consequences
 - **Pros:**
 - Get a more accurate model of the environment
 - Discover higher-reward states than the ones found so far
 - **Cons:**
 - When exploring, not maximizing your utility
 - Something bad might happen
- **Exploitation:** go with the best strategy found so far
 - **Pros:**
 - Maximize reward as reflected in the current utility estimates
 - Avoid bad stuff
 - **Cons:**
 - Might prevent you from discovering the true optimal strategy

Q-Learning: ϵ -Greedy Behavior Policy

Getting data with both **exploration** and **exploitation**

- With probability ϵ , take a random action; else the action with the highest (current) $Q(s, a)$ value.

$$a = \begin{cases} \operatorname{argmax}_{a \in A} Q(s, a) & \text{uniform}(0, 1) > \epsilon \\ \text{random } a \in A & \text{otherwise} \end{cases}$$

Q-learning Algorithm

Input: step size α , exploration probability ϵ

1. set $Q(s,a) = 0$ for all s, a .
2. For each episode:
3. Get initial state s .
4. While (s not a terminal state):
 - 5. Perform $a = \epsilon\text{-greedy}(Q, s)$, receive r, s'
 - 6. $Q(s, a) = (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$
Explore: take action to see what happens.
 - 7. $s \leftarrow s'$
Update action-value based on result.
8. End While
9. End For

Break & Quiz

Q 3.1 For Q learning to converge to the true Q function, we must

- A. Visit every state and try every action
- B. Perform at least 20,000 iterations.
- C. Re-start with different random initial table values.
- D. Prioritize exploitation over exploration.

Summary

- Reinforcement learning setup
- Mathematical formulation: MDP
- Bellman Equation
- Value Iteration Algorithm
- The Q-learning Algorithm